

TensorFlow Functions

Complete Notes

Definitions • Code Examples • Graphs/Diagrams • Why each function is used

Quick-revision notes for forgotten syntax & concepts

Table of Contents

- 1. Tensors & Basic Creation Functions
- 2. Variables (tf.Variable)
- 3. Math / Arithmetic Operations
- 4. Reduction Functions (sum, mean, max...)
- 5. Reshaping & Indexing Functions
- 6. Activation Functions (with graphs)
- 7. tf.function & AutoGraph (computation graph)
- 8. tf.GradientTape (Automatic Differentiation)
- 9. Loss Functions
- 10. Optimizers
- 11. tf.data.Dataset Pipeline Functions
- 12. Keras Layers & Model Building
- 13. Model Functions: compile / fit / evaluate / predict
- 14. Saving & Loading Models
- 15. Quick Revision Cheat-Sheet Table

1. Tensors & Basic Creation Functions

A **Tensor** is the core data structure of TensorFlow — an n-dimensional array (like a NumPy array) that can run on CPU/GPU/TPU. Almost everything in TensorFlow is a Tensor flowing through operations (hence the name 'TensorFlow').

tf.constant()

What it does: Creates an immutable tensor with fixed values that cannot be changed after creation.

Why we use it: Used to store fixed data such as inputs, fixed weights, or any value that should never change during the program, so TensorFlow can optimize it safely.

```
import tensorflow as tf

a = tf.constant([[1, 2], [3, 4]])
print(a)
# <tf.Tensor: shape=(2, 2), dtype=int32,
#   numpy=array([[1, 2], [3, 4]])>
```

tf.zeros() / tf.ones()

What it does: Creates a tensor filled entirely with zeros or ones of a given shape.

Why we use it: Commonly used to initialize biases (zeros) or masks/placeholders (ones) before training.

```
zeros = tf.zeros([2, 3])    # 2x3 tensor of 0s
ones  = tf.ones([3, 3])    # 3x3 tensor of 1s
```

tf.random.normal() / tf.random.uniform()

What it does: Generates a tensor with random values from a normal (Gaussian) or uniform distribution.

Why we use it: Used to randomly initialize weights of a neural network so that learning starts from a non-symmetric point (important for breaking symmetry between neurons).

```
w = tf.random.normal([3, 3], mean=0.0, stddev=1.0)
u = tf.random.uniform([2, 2], minval=0, maxval=1)
```

tf.cast()

What it does: Converts a tensor from one data type (dtype) to another, e.g. int32 to float32.

Why we use it: Used because many operations (like division, gradients) require float types, while raw data (e.g. images) is often loaded as int.

```
x = tf.constant([1, 2, 3])    # int32
x = tf.cast(x, dtype=tf.float32) # now float32
```

2. Variables (tf.Variable)

tf.Variable()

What it does: Creates a mutable tensor whose value can be changed/updated later — typically using .assign(), .assign_add(), or during optimizer updates.

Why we use it: Weights and biases of a neural network **MUST** be tf.Variable, because training works by repeatedly updating them via gradients. tf.constant cannot be updated.

```
w = tf.Variable(tf.random.normal([2, 2]))
w.assign(w + 1)    # update value
w.assign_add([[1,1],[1,1]]) # add in-place
```

3. Math / Arithmetic Operations

`tf.add()`, `tf.subtract()`, `tf.multiply()`, `tf.divide()`

What it does: Element-wise arithmetic operations between two tensors of the same (or broadcastable) shape.

Why we use it: These form the building blocks of every numeric computation in a model — same as `+`, `-`, `*`, `/` but tensor-aware and GPU-accelerated.

```
a = tf.constant([1, 2, 3])
b = tf.constant([4, 5, 6])
tf.add(a, b)      # [5 7 9]   (same as a + b)
tf.multiply(a, b) # [4 10 18] (element-wise)
```

`tf.matmul()`

What it does: Performs matrix multiplication (not element-wise) between two tensors — the core operation of every Dense/Linear layer: $\text{output} = X @ W + b$.

Why we use it: Neural network layers are mathematically matrix multiplications. `tf.matmul` is the function that actually computes the forward pass of a Dense layer.

```
X = tf.constant([[1., 2.]])      # shape (1,2)
W = tf.constant([[1.], [1.]])    # shape (2,1)
output = tf.matmul(X, W)         # shape (1,1)
```

Computation graph built by `@tf.function` (traced once, runs as fast C++ graph)

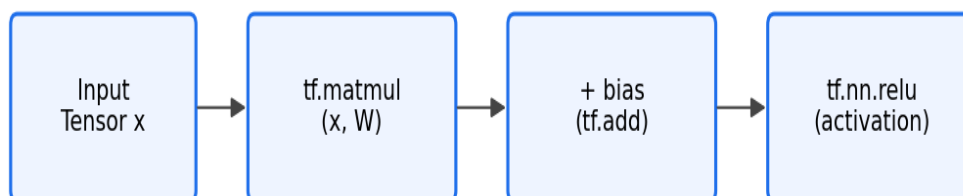


Fig 1: A simple computation graph — $x \rightarrow \text{matmul}(W) \rightarrow \text{add}(\text{bias}) \rightarrow \text{relu}$. This is exactly what happens inside a Dense layer.

4. Reduction Functions (sum, mean, max...)

`tf.reduce_sum()` / `tf.reduce_mean()` / `tf.reduce_max()`

What it does: Collapses (reduces) a tensor along given axis/axes by summing, averaging, or taking the max.

Why we use it: Used to compute total loss (reduce_sum/mean over a batch), pooling operations, and summarizing large tensors into single numbers or smaller tensors.

```
x = tf.constant([[1., 2.], [3., 4.]])
tf.reduce_sum(x)      # 10.0 (sum of all)
```

```
tf.reduce_mean(x, axis=0) # [2. 3.] (column means)
tf.reduce_max(x, axis=1) # [2. 4.] (row max)
```

5. Reshaping & Indexing Functions

tf.reshape()

What it does: Changes the shape of a tensor without changing its data (total number of elements stays the same).

Why we use it: Used constantly to flatten images before a Dense layer, or to convert between (batch, H, W, C) formats expected by different layers.

```
x = tf.range(12)           # shape (12,)
x = tf.reshape(x, [3, 4])  # shape (3,4)
x = tf.reshape(x, [-1])    # flatten back to (12,)
```

tf.expand_dims() / tf.squeeze()

What it does: expand_dims adds a new dimension of size 1; squeeze removes dimensions of size 1.

Why we use it: Used to add a 'batch' dimension before feeding a single image to a model, or remove an unwanted extra dimension after a prediction.

```
img = tf.zeros([28, 28])      # single image
img = tf.expand_dims(img, axis=0) # shape (1,28,28) batch of 1
img = tf.squeeze(img, axis=0)   # back to (28,28)
```

6. Activation Functions (with graphs)

Activation functions introduce non-linearity into a neural network. Without them, stacking layers would behave like one single linear layer, no matter how deep the network is.

tf.nn.relu()

What it does: Returns $\max(0, x)$ — outputs the input directly if positive, else outputs 0.

Why we use it: Most widely used activation in hidden layers because it is computationally cheap and avoids the vanishing-gradient problem better than sigmoid/tanh.

```
x = tf.constant([-3., -1., 0., 2., 5.])
tf.nn.relu(x) # [0. 0. 0. 2. 5.]
```

tf.sigmoid()

What it does: Squashes any real number into the range (0, 1) using $1 / (1 + e^{-x})$.

Why we use it: Used in the OUTPUT layer for binary classification, since its output can be interpreted directly as a probability.

```
tf.sigmoid(tf.constant([-5., 0., 5.])) # [~0.007, 0.5, ~0.993]
```

tf.nn.softmax()

What it does: Converts a vector of raw scores (logits) into a probability distribution that sums to 1.

Why we use it: Used in the OUTPUT layer of multi-class classification models (e.g. digit 0-9 classifiers) so each class gets a probability.

```
logits = tf.constant([2.0, 1.0, 0.1])
tf.nn.softmax(logits) # [0.66, 0.24, 0.10] (sums to 1)
```

tf.tanh()

What it does: Squashes input into the range $(-1, 1)$ — similar to sigmoid but zero-centered.

Why we use it: Often used inside RNN/LSTM cells because zero-centered outputs help gradients flow better than sigmoid.

```
tf.tanh(tf.constant([-2., 0., 2.])) # [-0.96, 0.0, 0.96]
```

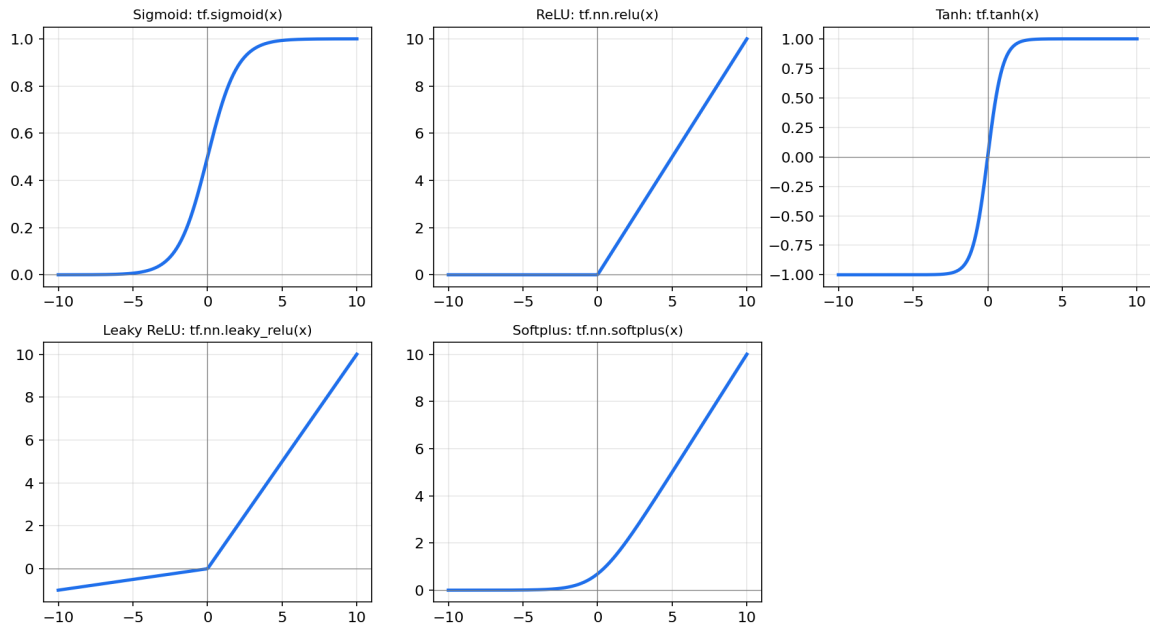


Fig 2: Shapes of the most common activation functions used in TensorFlow.

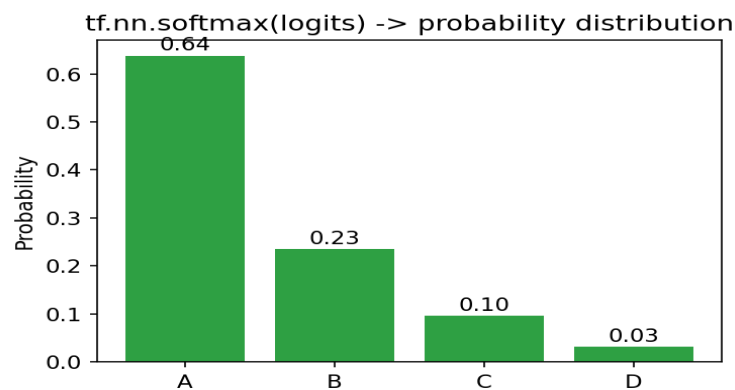


Fig 3: Softmax turns raw scores into a probability distribution.

7. tf.function & AutoGraph

@tf.function

What it does: A decorator that converts a normal (eager, slow, Python-level) function into a fast, optimized TensorFlow computation GRAPH, compiled once and reused on every call.

Why we use it: Used to speed up training/inference significantly — graph execution avoids Python overhead and lets TensorFlow optimize/parallelize operations, especially valuable in production.

```
@tf.function
def forward(x, w, b):
    return tf.nn.relu(tf.matmul(x, w) + b)

# First call traces the graph; later calls reuse it (fast)
output = forward(x, w, b)
```

Computation graph built by @tf.function (traced once, runs as fast C++ graph)

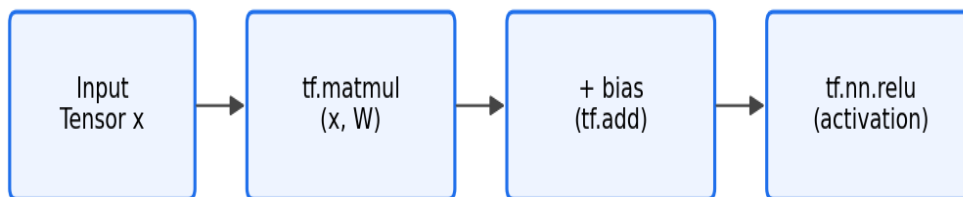


Fig 4: @tf.function traces Python code once into a static graph for repeated fast execution.

8. tf.GradientTape (Automatic Differentiation)

tf.GradientTape()

What it does: A context manager that 'records' every tensor operation performed inside it, so gradients (derivatives) of any result with respect to any watched variable can be computed automatically.

Why we use it: This is the heart of training a neural network — it computes $d\text{Loss}/d\text{Weight}$ for every weight so the optimizer knows which direction to adjust weights to reduce the loss.

```
w = tf.Variable(3.0)

with tf.GradientTape() as tape:
    loss = w ** 2          # any function of w

grad = tape.gradient(loss, w)  # dLoss/dw = 2w = 6.0
print(grad.numpy())          # 6.0
```

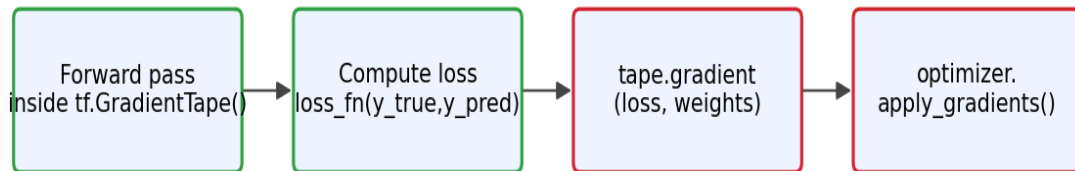
Typical full training step using GradientTape:

```
with tf.GradientTape() as tape:
    predictions = model(x_batch)
```

```
loss = loss_fn(y_batch, predictions)

grads = tape.gradient(loss, model.trainable_variables)
optimizer.apply_gradients(zip(grads, model.trainable_variables))
```

tf.GradientTape: automatic differentiation flow



*Tape 'records' every operation on watched tensors,
then replays them backward to compute $d\text{Loss}/d\text{Weight}$*

Fig 5: GradientTape records the forward pass, then computes gradients backward.

9. Loss Functions

`tf.keras.losses.MeanSquaredError()` (MSE)

What it does: Computes the average of the squared differences between predicted and true values.

Why we use it: Standard loss for REGRESSION problems (predicting continuous numbers like price or temperature).

```
mse = tf.keras.losses.MeanSquaredError()
loss = mse(y_true, y_pred)
```

`tf.keras.losses.BinaryCrossentropy()`

What it does: Measures the difference between two probability distributions for a 2-class problem.

Why we use it: Standard loss for BINARY CLASSIFICATION (e.g. spam vs not-spam), paired with a sigmoid output.

```
bce = tf.keras.losses.BinaryCrossentropy()
loss = bce(y_true, y_pred)
```

`tf.keras.losses.SparseCategoricalCrossentropy()`

What it does: Cross-entropy loss for multi-class classification where labels are given as plain integers (e.g. 0, 1, 2...) rather than one-hot vectors.

Why we use it: Standard loss for MULTI-CLASS CLASSIFICATION (e.g. classifying digits 0-9), paired with a softmax output layer.

```
scce = tf.keras.losses.SparseCategoricalCrossentropy()
loss = scce(y_true, y_pred)
```

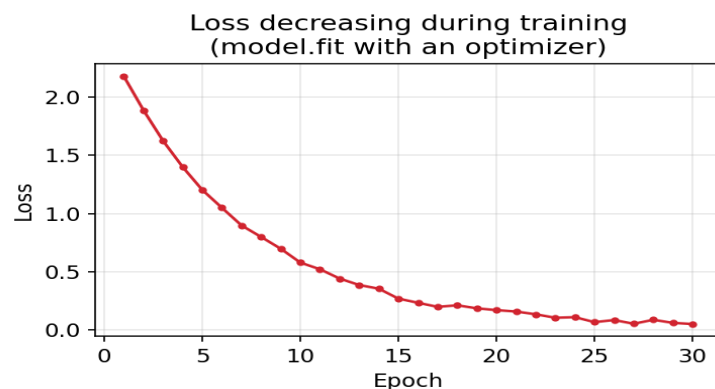


Fig 6: Loss decreasing across epochs as the optimizer updates weights.

10. Optimizers

`tf.keras.optimizers.SGD()`

What it does: Stochastic Gradient Descent — updates weights by moving a small step opposite the gradient: $w = w - \text{learning_rate} * \text{gradient}$.

Why we use it: The simplest and most fundamental optimizer; easy to understand and a good baseline.

```
opt = tf.keras.optimizers.SGD(learning_rate=0.01)
opt.apply_gradients(zip(grads, model.trainable_variables))
```

`tf.keras.optimizers.Adam()`

What it does: An advanced optimizer that adapts the learning rate for each parameter individually, combining momentum and RMSProp ideas.

Why we use it: The most commonly used optimizer in practice because it converges faster and more reliably than plain SGD on most deep learning problems.

```
opt = tf.keras.optimizers.Adam(learning_rate=0.001)
model.compile(optimizer=opt, loss="mse")
```

11. tf.data.Dataset Pipeline Functions

tf.data.Dataset.from_tensor_slices()

What it does: Creates a Dataset object from arrays/tensors, slicing them along the first dimension into individual elements.

Why we use it: Used to wrap raw NumPy/Tensor data into TensorFlow's efficient, GPU-friendly streaming pipeline instead of loading everything into memory at once.

```
ds = tf.data.Dataset.from_tensor_slices((x_train, y_train))
```

.map() / .shuffle() / .batch() / .prefetch()

What it does: map() applies a preprocessing function to every element; shuffle() randomizes order; batch() groups elements into mini-batches; prefetch() prepares the next batch while the current one is being processed by the GPU.

Why we use it: Together they build an efficient input pipeline so the GPU never sits idle waiting for data — critical for fast training on large datasets.

```
ds = (ds.map(preprocess_fn)
      .shuffle(1000)
      .batch(32)
      .prefetch(tf.data.AUTOTUNE))
```

tf.data.Dataset pipeline

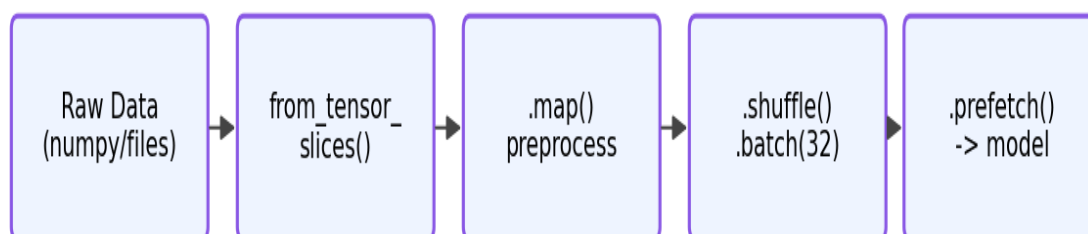


Fig 7: A typical tf.data input pipeline, step by step.

12. Keras Layers & Model Building

tf.keras.layers.Dense()

What it does: Creates a fully-connected layer: output = activation(input @ kernel + bias).

Why we use it: The most basic and most-used building block of neural networks for combining all input features.

```
layer = tf.keras.layers.Dense(units=64, activation="relu")
output = layer(input_tensor)
```

tf.keras.layers.Conv2D()

What it does: Applies a sliding convolution filter over a 2D input (like an image) to extract spatial features (edges, textures, shapes).

***Why we use it:** Essential for image-related tasks (CNNs) because it captures local spatial patterns far more efficiently than a Dense layer.*

```
conv = tf.keras.layers.Conv2D(filters=32, kernel_size=3, activation="relu")
output = conv(image_batch)  # shape (batch,H,W,channels)
```

tf.keras.Sequential()

What it does: Stacks layers linearly, one after another, into a single model object.

***Why we use it:** The simplest way to define a model when layers are applied in a straight-line sequence (input -> layer1 -> layer2 -> ... -> output).*

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])
```

13. Model Functions: compile / fit / evaluate / predict

model.compile()

What it does: Configures the model for training by specifying the optimizer, loss function, and metrics to track.

Why we use it: Must be called before training — it tells the model HOW it should learn (which optimizer) and HOW to measure error (which loss).

```
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
```

model.fit()

What it does: Trains the model on data for a given number of epochs, automatically running the forward pass, loss computation, backpropagation, and weight updates internally.

Why we use it: The main function used to actually TRAIN a Keras model without writing a manual GradientTape loop.

```
history = model.fit(x_train, y_train,
                   epochs=10,
                   batch_size=32,
                   validation_data=(x_val, y_val))
```

model.evaluate()

What it does: Computes the loss and metrics on test/validation data, without updating weights.

Why we use it: Used to check how well the trained model generalizes to unseen data.

```
loss, acc = model.evaluate(x_test, y_test)
```

model.predict()

What it does: Runs the model in inference mode on new input data and returns the predicted outputs.

Why we use it: Used after training, to actually use the model for making predictions on new data.

```
predictions = model.predict(x_new)
```

14. Saving & Loading Models

model.save() / tf.keras.models.load_model()

What it does: save() stores the full model (architecture + weights + optimizer state) to disk; load_model() restores it later.

Why we use it: Used so a trained model does not need to be retrained every time — it can be saved once and reused/deployed anywhere.

```
model.save("my_model.keras")
loaded = tf.keras.models.load_model("my_model.keras")
```

15. Quick Revision Cheat-Sheet Table

Function	One-line purpose
tf.constant()	Fixed, unchangeable tensor
tf.Variable()	Trainable / mutable tensor
tf.matmul()	Matrix multiplication
tf.reduce_sum/mean()	Collapse tensor along an axis
tf.reshape()	Change tensor shape
tf.nn.relu()	Hidden-layer activation, non-linearity
tf.nn.softmax()	Multi-class probability output
tf.sigmoid()	Binary probability output
@tf.function	Convert eager code to fast graph
tf.GradientTape()	Auto-differentiation for training
tf.keras.losses.*	Measure prediction error
tf.keras.optimizers.*	Update weights to reduce loss
tf.data.Dataset	Efficient input data pipeline
tf.keras.layers.Dense/Conv2D	Build network layers
model.compile/fit/evaluate/predict	Configure / train / test / use model

End of Notes — revise this cheat-sheet table first if short on time.